STAGE 3 REPORT · GROKKING PREDICTORS

Commutator Defect: Stage 3 Sanity Checks

Did the predictor maths survive contact with a real checkpoint? Four checks, one refactor, and one honest numerical finding.

Prepared for: Jonathan John

Thesis: “A Unified Benchmark of Grokking Predictors in Neural Networks”

Predictor 8 of 8 · Xu (2026) commutator defect D

Date: 4 November 2026 · Status: Stage 3 complete (not committed)

Verdict in one line

Checks 1, 3 and 4 **PASS**. Check 2 **FAIL** at epochs 6000 and 12000 — not because the formula is wrong (the 2-parameter toy matches $\sqrt{2.5}$ for both η values), but because at epoch 6000 the loss gradients are tiny (~7×10^{-6}) and D becomes noise-dominated when η is halved.

1. What Stage 3 was meant to do

Stage 1 froze the measurement rules in `context.md` before any D curve was seen. Stage 2 wrote the predictor file `06_Code/src/predictors/commutator_defect.py`. Stage 3 asks a simpler question: is the code doing the maths we think it is doing?

We deliberately did **not** wire the predictor into `unified_measurements.py`, `run_nanda_benchmark.py`, or `analyze_nanda_unified.py`. That is Stage 4. No Git commit was made.

2. Small refactor before the checks

The per-pair arithmetic was pulled into a public helper so the sanity script (and later Stage 4) can call one pair without re-drawing batches:

`defect_one_pair(theta0, grad_a_fn, grad_b_fn, eta) → {D, grad_norm_A, grad_norm_B}`

`compute_commutator_defect` now calls this helper inside its K = 5 loop. Batch drawing, seed 20261103, CPU float64, eval mode, and return keys are unchanged.

> **Regression lock**
>
> Before editing: ran epoch-6000 / seed 0 → `/tmp/cd_before.json`.
>
> After editing: same call → `/tmp/cd_after.json`.
>
> Result: the two dicts were **exactly identical** (every float). Median D = 82.74520277111803.

3. How real data was loaded

Training seed index 0 (grok epoch 12987). Split rebuilt exactly as Spectral / AGE do — not the unseeded loader inside `recompute_from_checkpoints`:

```
seed = int(np.load(seed_0/seed.npy)[0]) # value 0
torch.manual_seed(seed); np.random.seed(seed)
train_loader, _ = get_dataloaders(number=113, batch_size=int(0.3*113*113))
→ rebuilt train split has 3,830 examples ✓
```

Checkpoints used: epochs **1000, 6000, 12000** (exact matches on disk; 801 saved checkpoints under seed_0).

4. The four checks

Check	What we asked	Result
1. Same batch twice	A = B on epoch 6000 ⇒ D must be exactly 0	**PASS · D = 0.0**
2. Halve η	η = 1e–3 vs 5e–4; every |Δ|/D■ < 0.05	**FAIL · ep. 6000 & 12000**
3. Reproducible	Same seed twice; caller weights untouched	**PASS · exact match**
4. 2-param toy	|D – √2.5| < 1e–6 for η = 0.1 and 1e–3	**PASS · err ~ 1e–15**

4.1 Check 1 — same batch twice

On the epoch-6000 checkpoint, one batch of 512 was drawn with generator seed 20261103. Both `grad_a_fn` and `grad_b_fn` used that same batch. Result: **D = 0.0 exactly**. When A = B, the two step orders are identical, so the gap must vanish. PASS.

4.2 Check 2 — halving η (the failing check)

Theory says the leading term of ■θ_AB − θ_BA■ is O(η²). After dividing by ■η g_A■ · ■η g_B■, D should be nearly independent of η. We compared medians at η = 1e–3 and η = 5e–4 on three checkpoints:

Epoch	median (η=1e–3)	median (η=5e–4)	|D■–D■|/D■	Gate
1000	51.622	51.656	0.00066	**PASS**
6000	82.745	140.603	0.699	**FAIL**
12000	112.128	101.163	0.0978	**FAIL**

Diagnosis — not a missing-η bug

If the denominator forgot an η, D would scale as η or η² when η is halved. The 2-parameter toy (Check 4) gives the same D for η = 0.1 and η = 1e–3, so the normalisation in code is correct.

At epoch 6000, ■g■ ≈ 7×10^{-6} (weight L2 ≈ 65). Then ■η g■ is about 10^{-8}–10^{-9}. The true O(η²) gap sits near float noise; the measured numerator scales closer to O(η), so D blows up roughly like 1/η when η is reduced. Extra η sweep: 1e–3 → 82.7, 5e–4 → 140.6, 2.5e–4 → 277.8, 1e–4 → 675.5.

At epoch 1000, ■g■ ≈ 1×10^{-4} and the η-halving test passes cleanly. At epoch 12000, ■g■ ≈ 9×10^{-3}; some pairs are stable, a few are noisy, and the median shifts by ~9.8%.

Practical reading: the frozen probe η = 1e–3 is fine for the benchmark protocol, but the η-independence sanity check is unreliable on snapshots where the training loss gradient is nearly flat.

4.3 Check 3 — reproducibility

Two calls of `compute_commutator_defect` on epoch 6000 with the same seed produced identical dicts. After both calls, every parameter of the caller model still satisfied `torch.equal` against a pre-call snapshot, and stayed on the original device and dtype. PASS.

4.4 Check 4 — 2-parameter toy

float64, θ■ = (0, 0). H_A = diag(1, 3), c_A = (1, 0); H_B = [[2,1],[1,1]], c_B = (0, 1); g_X(θ) = H_X (θ – c_X).

Expected D = √2.5 ≈ 1.58113883 for both η = 0.1 and η = 1e–3. Observed errors were ~10^{-15} and ~10^{-14}. PASS.

For η = 0.1 the code gave θ_AB = (0.18, 0.09) and θ_BA = (0.19, 0.07). (The onboarding note's hand value (0.19, 0.04) for θ_BA was a slip; we print what the code computes and do not force it.)

5. Timing for Stage 5 planning

Average wall-clock over 8 `compute_commutator_defect` calls on this machine: ≈ **0.31 seconds per snapshot** (individual calls ~0.27–0.44 s). Rough single-threaded estimate for 2,005 snapshots (401 epochs × 5 seeds): **~0.17 hours**. That is cheap enough that Stage 5 is not blocked by compute cost on Apple Silicon.

6. Files involved

File	Role	Git status
06_Code/src/predictors/commutator_defect.py	Predictor + defect_one_pair refactor	untracked
06_Code/scripts/sanity_commutator_defect.py	Stage 3 runner (4 checks)	untracked
context.md	Stage 2 logged; Stage 3 not yet appended	modified
unified_measurements / run_nanda_benchmark / analyze_nanda_unified	Not touched (Stage 4)	unchanged

7. What this means for the thesis

Stage 3 does **not** score the predictor against grok epochs. It only asks whether the measurement is self-consistent. Three of four algebraic / engineering checks pass. The η-halving failure is a numerical caveat on near-flat-gradient snapshots, not evidence that the frozen Xu formula was coded incorrectly.

The frozen rules (η = 1e–3, K = 5, onset rule, PASS criteria) remain unchanged. Any later sensitivity study of η must be labelled secondary, as Stage 1 already said.

8. Next step

Stage 4 (not started)

Wire Commutator Defect into `unified_measurements.py`, `run_nanda_benchmark.py`, and `analyze_nanda_unified.py`.

Then Stage 5: run D on the saved checkpoint grid (≈ 401 × 5) and score against the frozen onset / PASS protocol.

Optional follow-up (secondary, not scored): document on which epochs ■g■ is too small for the η-independence check to be meaningful.

How to re-run: `cd …/grokking-benchmark && .venv/bin/python 06_Code/scripts/sanity_commutator_defect.py`

Companion note: `Grokbot outputs/Commutator_Defect_Explained.pdf` (Stage 0–1 onboarding).